第 04 章

LCEL出题、结构化输出与质量门禁

模型生成内容，后端绑定参考原文：用两条固定 Chain 完成可检查的个人题库。

这一章完成后，你能够

- 区分 LCEL 固定链、Celery 作业与辅导 Agent
- 读懂模型内容 Schema 与后端来源 Schema
- 解释分组取材、字符硬预算、基础拒绝与一次补充
- 运行离线实验，观察来源绑定和题干重复的处理结果

学习前提

完成资料解析与切块章节；理解 Python 列表、字典、类型标注和 UUID；能够使用项目已锁定依赖的 Python 环境运行脚本

代码基准：8aa0ba5 + 本轮名称与分页修改。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

4.1 业务场景：资料解析完成以后，题目从哪里来? 3

4.2 名词解析：LCEL、Runnable 和 Parser 如何配合? 4

4.3 原文分组：更分散的覆盖与明确的字符预算 5

4.4 知识点链：内容由模型返回，来源由后端记录 7

4.5 题目链：结构检查与参考原文绑定 9

4.6 拒绝规则：保留基础检查，移除模型引文匹配 10

4.7 保存与界面：为什么会出现 partial 和待审核? 12

4.8 完整离线实验：先解析内容，再绑定实际来源 15

4.9 动手练习：把规则、计数与语义分开解释 18

4.10 综合练习与参考思路 19

4.11 源码阅读清单与下一步 20

4.1 业务场景：资料解析完成以后，题目从哪里来?

上传资料后，学习者需要可作答的题目、标准答案和能回到原文的阅读入口。旧版要求模型同时填写片段 ID 与逐字引文，容易在题目内容看似合理时因来源格式失败。当前方案 B 把这项机械工作交给后端，模型专心返回题干、答案与解析。

一次生成请求包含目标题数、允许题型、难度范围和语言。应用服务先确认资料已经 ready、知识库有效且没有冲突任务，再创建 pending 作业并投递 Celery。后台 Worker 执行固定步骤；HTTP 请求无需一直等待模型完成，也不会让模型自行决定是否把结果写入数据库。



图 4-1 程序控制顺序；每组执行两类链，模型不选择工具或写业务库



图 4-2 Vue 题库配合隔离教学响应：观察独立题目对象、待审核状态与参考片段入口。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。演示待审核题和两个来源类型；示例计数不作为质量指标

本章观察任务

先找出待审核题与已启用题，再思考：为什么模型返回一道题以后，还需要校验和人工启用？把“生成完成”与“能够开始练习”分开理解。

4.2 名词解析：LCEL、Runnable 和 Parser 如何配合?

LCEL 是 LangChain Expression Language，即用表达式组合可执行组件的方式。这里的竖线符号 | 表示把上一步输出交给下一步：Prompt 生成消息，模型返回内容，Parser 把内容转换成业务模型。三个组件组成可调用的 Runnable 链，程序通过 invoke 或 ainvoke 执行它。

名词 / 函数	直观含义	本项目用途
ChatPromptTemplate	含变量的聊天消息模板	分别规定抽取与出题的角色、原文和约束

名词 / 函数	直观含义	本项目用途
partial	提前固定部分模板变量	注入格式说明，调用时不用再次传入
Runnable	可统一调用的执行组件	以 dict 为输入，以知识点或题目批次为输出
PydanticOutputParser	把模型文本解析成类型对象	执行字段长度、取值和题型结构校验
with_structured_output	配置模型的结构化返回方式	在配置选择非 parser 路径时使用
Celery Task	异步后台作业	负责阶段推进、状态记录和任务级重试

表 4-1 把术语落实到本章的真实作用。

不要把“链中使用了模型”直接理解为 Agent。当前出题顺序由 tasks.py 写定，模型只能产生知识点和题目候选；它没有工具选择循环。辅导 Agent 才会在约束范围内调用资料搜索、进度和错题工具。Celery、LCEL 和 Agent 是不同层面的机制，可以同时存在于一个项目。

阅读类型标注 `Runnable[dict[str, Any], QuestionBatch]` 时，可以把方括号两侧当成接口合同：调用方传入字典，成功以后收到 QuestionBatch 对象。这样下游读取 items，不必猜模型是否返回 Markdown、列表或解释文字。但成功解析只证明结构满足要求，还不能证明答案有充分依据。

4.3 原文分组：更分散的覆盖与明确的字符预算

Worker 从 PostgreSQL 读取片段，plan_generation_groups 会过滤空正文并按 chunk_index 排序。组数是 3、目标题数、ceil(可用块数 / 4) 三者的最小值；因此少量资料或只请求一题时不一定有三组。每组窗口最多四块，起点从头到尾均匀分布，组内仍按原文顺序拼接。

源码节选：组数、窗口与题数分配

定位：apps/api/study_agent/question_generation/context.py，第 35-58 行。

```
    group_count = min(
        MAX_CONTEXT_GROUPS,
        target_count,
        (len(available) + MAX_CHUNKS_PER_GROUP - 1) // MAX_CHUNKS_PER_GROUP,
    )
    if group_count < 1 or max_chars < 100 * group_count:
        raise ValueError("出题数量或上下文预算过小")
    width = min(MAX_CHUNKS_PER_GROUP, len(available))
    quota, remainder = divmod(target_count, group_count)
    groups: list[GenerationGroup] = []
    for index in range(group_count):
        start = (
            round(index * (len(available) - width) / (group_count - 1)) if group_count > 1 else 0
        )
        budget = max_chars // group_count
        selected: list[DocumentChunk] = []
```

代码续

```
        sections: list[str] = []
        for chunk in available[start : start + width]:
            heading = " / ".join(chunk.heading_path) or "无标题"
            header = f"[标题={heading}; 页码={chunk.page_start or '未知'}]\n"
            remaining = budget - len(header) - (2 if sections else 0)
            if remaining <= 0:
                break
            content = chunk.content[:remaining]
```

target_count 是整个作业的目标数；max_chars 是全部组共同的字符预算。

例如 40 块、目标 10 题：三组从索引 0 / 18 / 36 开始，各组目标 4 / 3 / 3。

参数 / 名词	当前实现	应怎样理解
MAX_CONTEXT_GROUPS	3	最多三组，不是遍历所有章节
MAX_CHUNKS_PER_GROUP	4	每组最多四块；预算较小时还会更少
target_count	整除分配，余数给前面的组	10 题分为 4 / 3 / 3
max_chars 默认值	24000 字符，含标题和分隔符	全部组输入总长的上限，不等于 Token 数
长片段	按剩余预算截断正文	绑定同一截断文本，不保存未提供的尾部
模型可见头部	标题 / 页码；无数据库 UUID	页码未知时明确标为“未知”

表 4-2 分组函数的默认限制；短材料可能只需要一组

源码节选：截断正文并保存实际组输入

定位：apps/api/study_agent/question_generation/context.py，第 56-74 行。

```
            if remaining <= 0:
                break
            content = chunk.content[:remaining]
            section = header + content
            selected.append(replace(chunk, content=content))
            sections.append(section)
            budget -= len(section) + (2 if len(sections) > 1 else 0)
            if len(content) < len(chunk.content):
                break
        if not selected:
            raise ValueError("原文标题过长，无法在上下文预算内出题")
        groups.append(
            GenerationGroup(
                chunks=selected,
                context="\n\n".join(sections),
                target_count=quota + (index < remainder),
            )
        )
    return groups
```

每组分得 floor(max_chars / group_count) 字符，剩余尾数不再分配。

replace(chunk, content=content) 保留 ID 等定位元数据，同时把正文限定为实际送入模型的文本。



图 4-3 把“原文存了多少”与“本次模型读了多少”区分开

取舍与边界

这仍然是材料驱动抽样，没有 Milvus 查询召回。总字符预算只统计 group.context，不包含系统提示、Schema 和知识点 JSON；补充一轮会再次使用已有上下文，因此不能把 24000 解读为整个作业累计请求字符。

分散窗口比只读前部更容易接触后面的内容，但缺少章节覆盖审计，无法承诺整份资料每节都考到了。教材必须保留这一限制。后续若真要保证覆盖，可按章节切任务，再检查每章有效题数；个人项目当前采用小组抽样来控制复杂度和调用量。

4.4 知识点链：内容由模型返回，来源由后端记录

KnowledgePointDraft 只包含 canonical_key、name、description、importance 和 difficulty。模型不填写 source_chunk_ids。canonical_key 是题目引用知识点的稳定键，例如 tcp-handshake；它不是数据库 ID。链的返回类型仍是 KnowledgePointBatch，items 有 1-30 项的结构约束。

源码节选：知识点提示词与解析器

定位：apps/api/study_agent/question_generation/chains.py，第 17-42 行。

```
17  def create_knowledge_point_chain(
18      settings: Settings,
19  ) -> Runnable[dict[str, Any], KnowledgePointBatch]:
20      parser = PydanticOutputParser(pydantic_object=KnowledgePointBatch)
21      prompt = ChatPromptTemplate.from_messages(
22          [
23              (
24                  "system",
25                  "你是职业学习资料分析器。只根据提供的原文片段提取可考查知识点。"
26                  "canonical_key 应稳定、简短、可用于去重。来源由系统绑定，不要输出片段 ID 或引文。",
```

代码续

```
            ),
            (
                "human",
                "语言：{language}\n本组出题参考原文：\n{context}\n\n"
                "输出要求：\n{format_instructions}",
            ),
        ]
    ).partial(format_instructions=parser.get_format_instructions())
    model = get_chat_model(settings, purpose="question_generation")
    if settings.llm_structured_output_method == "parser":
        chain = prompt | model | parser
    else:
        chain = prompt | model.with_structured_output(
            KnowledgePointBatch, method=settings.llm_structured_output_method
        )
    return cast(Runnable[dict[str, Any], KnowledgePointBatch], chain)
```

输入 language 与本组 context，format_instructions 提前固定。

默认 parser 分支使用 prompt | model | parser；其他配置分支使用模型的结构化接口。

对象 / 函数	输入与输出	为什么要分开
KnowledgePointDraft	模型输出：键、名称、描述、重要度、难度	减少模型需要维护的机械字段
KnowledgePointBatch	本组的一组知识点	供本组出题链选择关联键
SourcedKnowledgePointDraft	后端追加本组 source_chunk_ids	保存关系时使用真实输入片段
SourcedKnowledgePointBatch	合并各组的带来源知识点	在任务中统一保存与复用
serialize_knowledge_points	模型内容 Schema → JSON	只把本组知识点内容送给出题链

表 4-3 模型输出合同与后端持久化合同各有职责

任务对每组调用提取链，再用该组实际 chunks 构造 SourcedKnowledgePointDraft。一个知识点绑定本组多个块，含义是“本组原文用于提取这个知识点”，不表示每个块都逐句支持该描述。保存时规范化 canonical_key，并复用知识库已有知识点与来源关系。

不要夸大来源关系

后端确保片段来自本次实际输入，但不会自动证明模型描述的每句话都正确。知识点和题目的语义支持仍需要阅读参考原文来确认。

4.5 题目链：结构检查与参考原文绑定

出题链收到本组原文、本组知识点、题数、允许题型、难度、语言和已有题干。GeneratedQuestionDraft 没有来源字段，模型只填写知识点键、题型、题干、选项、答案、解析和难度。已有题干帮助模型避免重复，确定性程序仍会再次检查。

源码节选：题型结构验证

定位：apps/api/study_agent/question_generation/schemas.py，第 49-71 行。

```python
    def validate_type_shape(self) -> "GeneratedQuestionDraft":
        if self.question_type == "single_choice":
            if self.options is None or len(self.options) != 4:
                raise ValueError("单选题必须包含 A、B、C、D 四个选项")
            if {option.key for option in self.options} != {"A", "B", "C", "D"}:
                raise ValueError("单选题选项键必须为 A、B、C、D")
            if len(self.correct_answers) != 1 or self.correct_answers[0] not in {
                "A",
                "B",
                "C",
                "D",
            }:
                raise ValueError("单选题必须且只能有一个正确选项")
        elif self.question_type == "true_false":
            self.options = None
            if len(self.correct_answers) != 1 or self.correct_answers[0] not in {
                "正确",
                "错误",
            }:
                raise ValueError("判断题答案必须为正确或错误")
        else:
            self.options = None
        return self
```

单选必须有 A-D 四个键和一个合法答案；判断答案只能是“正确”或“错误”。

判断和填空把 options 归为 None；选项文字是否重复留给基础业务校验。

题型 / 对象	结构与职责	例子
single_choice	四个选项，一个正确键	correct_answers=["C"]
fill_blank	options=None，允许答案别名	["三次", "3次"] 不表示两个空
true_false	答案仅“正确”或“错误”	不以 true 或 1 作为生成标准答案
GeneratedQuestionDraft	模型题目内容，不请求 UUID 或引文	Schema 成功不代表答案语义正确
SourcedQuestionDraft	后端追加 IDs、文本、source_kind	source_kind="reference_context"

表 4-4 题型结构保持严格，来源机械工作交给后端

源码节选：把实际输入原文绑定到题目

定位：apps/api/study_agent/question_generation/context.py，第 78-86 行。

```python
def bind_question_context(
    question: GeneratedQuestionDraft, group: GenerationGroup
) -> SourcedQuestionDraft:
    # Construct from the parsed content schema, overriding any unsolicited model sources.
    return SourcedQuestionDraft(
        **question.model_dump(),
        source_chunk_ids=[chunk.id for chunk in group.chunks],
        source_quotes=[chunk.content for chunk in group.chunks],
    )
```

question 是已经解析的模型内容对象，group 是本组实际输入。

source_chunk_ids 和 source_quotes 均来自 group.chunks；模型额外返回的来源字段不会成为正式来源。

source_quotes 这个兼容字段名仍在使用，但新记录装的是出题上下文，而不是模型逐字摘录的答案引文。每题绑定本组最多四块实际原文，可能比答案直接依据更宽。API 还返回资料标题、页码与标题路径，页面可折叠展开，方便人工查看。

source_kind	保存与展示	阅读时的判断
quote	旧记录保留“原文引文”	不自动改写为新版上下文
reference_context	新记录展示“出题上下文”	说明模型看到的本组参考原文
迁移 0009	历史记录默认仍为 quote	新增检查约束只允许两种取值

表 4-5 题库和作答讲解使用同一来源类型区分；重复题保留已有旧引文

4.6 拒绝规则：保留基础检查，移除模型引文匹配

当前门禁分为三层。第一层 Parser / Pydantic 检查 JSON 和题型结构，出错通常进入任务异常与有限重试；第二层 validate_question_drafts 逐题保存首个拒绝原因；第三层持久化检查已删除的相同题。结构错误与逐题 rejected_count 不是同一种统计。

源码节选：请求范围与题干重复校验

定位：apps/api/study_agent/question_generation/quality.py，第 37-66 行。

```python
    for question in questions:
        reason: str | None = None
        if question.question_type not in allowed_types:
            reason = "题型不在请求范围内"
        elif question.knowledge_point_key not in point_keys:
            reason = "知识点不存在"
        elif not difficulty_min <= question.difficulty <= difficulty_max:
            reason = "难度超出请求范围"
        elif any(chunk_id not in chunk_map for chunk_id in question.source_chunk_ids):
```

代码续

```
            reason = "引用了不存在的片段"
        normalized_stem = _normalize(question.stem)
        if reason is None and normalized_stem in seen_stems:
            reason = "同批次题干重复"
        if reason is None and question.question_type == "single_choice":
            option_texts = {_normalize(option.text) for option in question.options or []}
            if len(option_texts) != 4:
                reason = "单选题选项重复"
        if (
            reason is None
            and question.question_type == "fill_blank"
            and any(len(answer) > 100 for answer in question.correct_answers)
        ):
            reason = "填空题答案过长"

        if reason:
            rejected.append(RejectedQuestion(question=question, reason=reason))
        else:
            seen_stems.add(normalized_stem)
            accepted.append(question)
    return accepted, rejected
```

检查顺序决定展示哪个原因；一题同时触发多条时只记录最先失败的一条。

seen_stems 由任务共享，检查本次作业的不同组和补充轮；不自动查询历史题干集合。

逐题拒绝类型	触发条件	说明
题型不在请求范围内	结构合法但不在 allowed_types	例如只要单选却返回判断
知识点不存在	knowledge_point_key 不属于本组	题目不能关联任意自造的键
难度超出请求范围	在全局 1-5 内，却超出本次范围	请求 1-3，返回 4
引用了不存在的片段	内部来源 ID 不属于提供的 chunks	后端正常绑定后不应发生
同批次题干重复	去空白、转小写后的题干已见过	包含跨组和补充轮
单选题选项重复	四个选项文字规范化后不足四种	只检查重复，不判断选项语义
填空题答案过长	任意标准答案超过 100 字符	保持个人题库答案简洁
相同题目已被删除	持久化发现相同哈希且 status=deleted	不让重新生成悄悄恢复已删除题

表 4-6 当前代码的基础拒绝原因；最后一条发生在保存阶段

旧规则为什么不再使用

“片段与引文数量不一致”和“引文无法在原文片段中匹配”是旧版模型填写来源时的拒绝原因。新版 IDs 与文本由后端同时生成，不再逐字匹配模型引文。历史作业仍可能显示旧原因，更新代码不会重新判定已经拒绝的候选。

这里没有引入另一位模型评审，也没有自动语义判题。模型可能生成结构正确但答案错误的题，基础门禁仍可能通过。减少来源格式误拒不等于取消人工内容审核；题库详情中读题干、标准答案、解析和参考原文后再启用。

4.7 保存与界面：为什么会出现 partial 和待审核?

每组先校验候选，再按该组目标截取有效题。合格但超出该组请求数量的候选属于正常截取，不计为拒绝；被截去的题干仍进入本作业 seen_stems。持久化计算题型、规范化题干与答案的内容哈希，复用有效旧题，并保留被删除题的拒绝说明。

阶段 / 状态	界面含义	核对位置
extracting_knowledge_points	正在抽取知识点；progress=10	是否已读到资料片段
generating_questions	正在生成候选；progress=45	结构化输出是否可解析
persisting	开始保存通过题；progress=75	来源、内容哈希与数据库关系
completed	保存计数达到目标；progress=100	不等于人工审核全部通过
partial	有效保存计数少于目标；progress=100	查看拒绝原因或候选数量不足
draft / active	待审核题 / 可用于学习的启用题	在题库中复核后启用

表 4-7 进度数值是固定阶段标记，不是模型完成百分比预测。



图 4-4 2026-10-09 隔离教学响应：生成阶段固定显示 45%，不是已完成题数比例。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。固定生成阶段 45%；展示阶段进度，不是已生成题数比例



图 4-5 Vue 题目详情配合隔离教学响应：展开新出题上下文和旧原文引文，按资料标题、页码与标题路径人工复核。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。新出题上下文与旧原文引文分别展开，包含标题、页码和标题路径

来源关系与题目保存之后，任务还会写入题库向量索引并再次检查题目是否在索引期间被删除。生成记录写明 execution_kind="chain"，有知识点和题目两个步骤摘要。历史数据库字段保留 agent_type 等名字，是兼容记录格式，不能据此把当前执行流程解释成 Agent。

计数的准确含义

generated_count 是保存或复用后的数量，不一定都是新增题；目标不足可能来自内容不足或保存阶段删题拦截，不一定意味着拒绝很多。补充轮会让总候选数超过用户目标。

首轮校验后的有效数不足时，任务只向第一组补充一次，传入已经接受的题干。若这次额外调用异常，首轮有效题仍保存，状态可为 partial，并提示“补充生成未完成，已保留本轮有效题目”。保存阶段才发现的删题拒绝不会再开启补充循环。



图 4-6 这是有界的一次补充，不是 Agent 自主反思或无限修复

45% 到底表示什么

10 / 45 / 75 / 100 是代码写死的阶段标记：提取知识点、生成候选、保存、处理结束。45% 不代表 10 题已经完成 4.5 题，不预测剩余时间；partial 结束也可显示 100%，failed 不强制把此前进度改成 100%。



图 4-7 题型、难度与目标题数的当前配置界面；只展示，不发送生成请求。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。展示新的后端绑定、一轮补充说明；没有提交生成

4.8 完整离线实验：先解析内容，再绑定实际来源

下面两个实验从项目根目录使用 apps/api/.venv/Scripts/python.exe 执行。FakeListChatModel 只返回固定 JSON，不调用外部模型或写业务数据库。第一个实验故意让模型带上错误来源额外字段，再构造重复题干，观察两种问题由哪一层处理。

完整可运行实验：LCEL、后端绑定与重复题拒绝

```python
import json
from uuid import UUID
from langchain_core.language_models.fake_chat_models import FakeListChatModel
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from study_agent.domain.models import DocumentChunk
from study_agent.question_generation.context import (
    bind_question_context, plan_generation_groups,
)
from study_agent.question_generation.schemas import (
```

代码续

```
    KnowledgePointDraft, QuestionBatch,
)
from study_agent.question_generation.quality import validate_question_drafts

chunk = DocumentChunk(
    UUID(int=1), UUID(int=2), 0, "TCP 通过三次握手建立连接。",
    16, 1, 1, ["网络", "TCP"], "offline-demo", None, None,
)
point = KnowledgePointDraft(
    canonical_key="tcp-handshake", name="TCP 三次握手",
    description="理解 TCP 建立连接的过程", importance=0.9, difficulty=2,
)
item = dict(
    knowledge_point_key=point.canonical_key, question_type="true_false",
    stem="TCP 通过三次握手建立连接。", correct_answers=["正确"],
    explanation="这一陈述与参考原文一致。", difficulty=2,
    source_chunk_ids=[str(UUID(int=999))], source_quotes=["伪造来源"],
)
parser = PydanticOutputParser(pydantic_object=QuestionBatch)
prompt = ChatPromptTemplate.from_messages([
    ("system", "只按原文出题。{format_instructions}"),
    ("human", "本组原文：{context}"),
]).partial(format_instructions=parser.get_format_instructions())
model = FakeListChatModel(responses=[json.dumps({"items": [item, item]})])
group = plan_generation_groups([chunk], target_count=2, max_chars=1000)[0]
batch = (prompt | model | parser).invoke({"context": group.context})
bound = [bind_question_context(q, group) for q in batch.items]
accepted, rejected = validate_question_drafts(
    knowledge_points=[point], questions=bound, chunks=[chunk],
    allowed_types={"true_false"}, difficulty_min=1, difficulty_max=3,
)
print("parsed =", len(batch.items))
print("accepted =", len(accepted), "rejected =", len(rejected))
print("reason =", rejected[0].reason)
print("source_kind =", accepted[0].source_kind)
assert "source_chunk_ids" not in batch.items[0].model_dump()
assert accepted[0].source_chunk_ids == [chunk.id]
assert accepted[0].source_quotes == [chunk.content]
assert len(accepted) == 1 and len(rejected) == 1
```

Parser 先得到不含来源的 QuestionBatch；bind_question_context 再构造带真实来源的候选。

两个题干相同，第一题接受、第二题拒绝；拒绝原因来自内容去重，而非模型引文。

结果解释：parsed = 2，accepted = 1 rejected = 1；reason = 同批次题干重复；source_kind = reference_context。



图 4-8 真实生产 Schema 与绑定函数，加固定模型返回值；不是模型准确率实验

第二个实验构造 40 个长片段，设置总预算 1500。三组窗口起点为 0 / 18 / 36，目标为 4 / 3 / 3；每个组最多 500 字符，所以第一个 800 字符正文也被截断。绑定文本仍完全包含在组上下文中，模型看不到原片段 UUID。

完整可运行实验：硬预算与截断文本绑定

```python
from dataclasses import replace
from uuid import UUID
from study_agent.domain.models import DocumentChunk
from study_agent.question_generation.context import (
    bind_question_context, plan_generation_groups,
)
from study_agent.question_generation.schemas import GeneratedQuestionDraft

first = DocumentChunk(UUID(int=1), UUID(int=2), 0, "甲" * 800, 800,
                      1, 1, ["第一节"], "demo", None, None)
chunks = [replace(first, id=UUID(int=i+1), chunk_index=i,
                  page_start=i+1, page_end=i+1) for i in range(40)]
groups = plan_generation_groups(chunks, target_count=10, max_chars=1500)
question = GeneratedQuestionDraft(
    knowledge_point_key="test-point", question_type="true_false",
    stem="这是一道测试判断题。", correct_answers=["正确"],
    explanation="依据本组原文作答。", difficulty=2,
)
bound = bind_question_context(question, groups[1])
print("starts =", [g.chunks[0].chunk_index for g in groups])
print("targets =", [g.target_count for g in groups])
print("within_budget =", sum(len(g.context) for g in groups) <= 1500)
print("uuid_hidden =", all(str(c.id) not in g.context
                           for g in groups for c in g.chunks))
print("truncated =", len(groups[0].chunks[0].content) < len(first.content))
print("bound_visible_text =", all(q in groups[1].context for q in bound.source_quotes))
assert sum(g.target_count for g in groups) == 10
assert sum(len(g.context) for g in groups) <= 1500
```

replace 用于构造独立教学片段；页面信息仍保留真实窗口位置。

比较的预算是全部 group.context 的总字符数；它不是模型完整请求和补充轮的累计用量。

结果解释：within_budget、uuid_hidden、truncated、bound_visible_text 均为 True。

实验能够证明什么

离线结果能证明 Schema、窗口选择、绑定和基础校验行为，不能证明模型生成的标准答案正确。2026-10-09 的独立供应商验收曾在合成资料上取得目标 10、通过 10、拒绝 0；这是单份样本的历史结果，本次更新 PDF 没有重新调用供应商。

4.9 动手练习：把规则、计数与语义分开解释

先尝试把第二题题干改成另一个表述，观察精确题干去重不再触发；再把难度改成 4，但请求范围仍是 1-3。最后故意把一道判断题的标准答案写反，说明为什么结构和基础规则都可能通过。这些练习把“来源真实”和“答案正确”拆成不同观察对象。

改动	预计现象	应检查的层次
伪造额外模型来源	正式绑定仍使用本组真实原文	内容 Schema 与后端 binder
两道相同题干	第二道记录重复原因	shared seen_stems
请求 1-3，返回 4	基础规则拒绝	difficulty 请求约束
答案写反但格式合法	可能仍通过	人工语义审核
减少字符预算	每组正文更短或报预算过小	组上下文总长与可见来源

表 4-8 先写预测，再运行实验；不要把结构通过率写成语义准确率



图 4-9 当前拒绝详情界面；隔离教学候选用于展示“单选题选项重复”，不是一次真实模型失败记录。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。当前基础拒绝示例：单选题 A 与 B 重复；候选未进入题库

复盘句式

我把来源机械字段从模型输出移到后端，保留结构、范围和重复检查；通过固定实验验证绑定与预算，再用人工阅读检查答案是否被参考原文支持。没有对照统计时，不写误拒率下降百分比。

4.10 综合练习与参考思路

练习 1：新版为什么无需模型输出引文?

参考思路：后端保存每组实际提供的片段及正文；模型只负责内容，避免在 UUID 与逐字摘录上出错。

练习 2：参考原文存在能证明答案正确吗?

参考思路：不能；来源说明模型的实际输入，可能比直接依据更宽，答案和解析仍需人工阅读核实。

练习 3：24000 字符限制什么?

参考思路：全部组上下文总长，包含标题与分隔符；不包含完整提示词或补充轮累计输入。

练习 4：目标 10 题却只得到 8 题，会无限重试吗?

参考思路：首轮有效数不足时只补充一次；额外调用失败仍保留首轮，保存后可能是 partial。任务异常的有限重试是另一层机制。

练习 5：为什么 100% 仍可能 partial?

参考思路：进度是阶段标记；100% 表示处理结束，partial 表示保存数量不足，并不表示题目已启用。

4.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/question_generation/context.py	GenerationGroup / plan_generation_groups / bind_question_context	窗口、字符预算与实际来源如何对齐
apps/api/study_agent/question_generation/chains.py	create_knowledge_point_chain / create_question_generation_chain	比较两类链的输入与内容 Schema
apps/api/study_agent/question_generation/schemas.py	GeneratedQuestionDraft / SourcedQuestionDraft	区分模型合同和后端绑定合同
apps/api/study_agent/question_generation/tasks.py	generate_questions / QuestionGenerationTask	阶段进度、一次补充、保存与有限重试
apps/api/study_agent/question_generation/quality.py	validate_question_drafts	列出当前逐题拒绝规则
apps/api/migrations/versions/0009_出题参考原文.py	upgrade / source_kind	旧 quote 与新 reference_context 如何共存
apps/api/tests/test_question_generation.py	test_model_sources_cannot_override_backend_context	用离线案例核对模型来源不进入正式记录

下一章：下一章转入辅导 Agent，比较模型选择工具的动态循环与本章固定出题步骤。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。